



Data scientist technical challenge

Prevención de fraude en pagos

Fraud Prevention Fintech, Mercado Libre

Challenge técnico de Data Scientist

Se entrena un modelo de machine learning para predecir fraude sobre un flujo de pagos proporcionado con features anonimizadas. El criterio de éxito es la ganancia: cada operación legítima aprobada deja un 25 % de su monto y cada fraude aprobado pierde el 100 %. Todo el trabajo, desde el umbral de decisión hasta la elección del modelo final, se ordena alrededor de esa función objetivo.

Carlos Federico Trejo

Informe de la solución propuesta

Código fuente y pasos de ejecución en el repositorio adjunto

Índice

1. Hipótesis y planteo económico	2
2. Análisis y transformaciones del dataset	2
2.1. Calidad de datos, faltantes y leakage	2
2.2. Distribuciones, outliers y la transformación que cada modelo necesita	3
2.3. La categórica de alta cardinalidad	3
2.4. Análisis temporal	4
3. Esquema de validación	4
3.1. Cómo se agregan las métricas entre folds	4
4. Modelos utilizados	6
4.1. Baseline: políticas y regresión logística	6
4.2. Modelos de árbol y búsqueda de hiperparámetros	7
4.3. Desbalance, reponderación y calibración	8
5. Robustez y selección del modelo final	8
6. Interpretabilidad	10
7. Evaluación final	11
8. Llevar el modelo a producción - Preguntas 3, 4 y 5	12
8.1. ¿Qué pasos puedo seguir para intentar asegurar que la performance del modelo en laboratorio será similar a la de producción?	12
8.2. Suponiendo que la performance predictiva en producción es muy diferente a la esperada, ¿cuáles cree que son las causas más probables?	13
8.3. ¿Qué pasos debería seguir para poner el nuevo modelo en producción?	13
9. Conclusión	13
A. Material complementario	15

1. Hipótesis y planteo económico

El problema pide predecir si una transacción es fraudulenta en donde el objetivo real es económico: maximizar la ganancia. Una operación legítima que aprobamos deja un 25 % de su monto, un fraude que aprobamos pierde el 100 %. Esa asimetría ordena las decisiones del proyecto y vale la pena hacerla explícita antes de tocar los datos.

La primera consecuencia es que la accuracy no sirve como métrica. Con una tasa de fraude cercana o igual al 5 %, aprobar todo acierta el 95 % sin haber aprendido nada útil. La métrica de comparación es la ganancia total, que pondera cada error por el monto de la operación, no por su cantidad. Rechazar un fraude de monto alto vale mucho más que rechazar uno chico, y eso un conteo de aciertos no lo ve.

La segunda consecuencia es el umbral de decisión. No hay razón para usar 0,5. Si un modelo estima una probabilidad de fraude p para una operación de monto M , la ganancia esperada de aprobarla es

$$\mathbb{E}[G_{\text{aprobar}}] = (1 - p) 0,25 M - p M = M [0,25(1 - p) - p]. \quad (1)$$

Conviene aprobar mientras esa cantidad sea positiva, lo que despejando da

$$0,25 - 1,25 p > 0 \iff p < \frac{0,25}{1,25} = 0,20. \quad (2)$$

El corte de decisión es **20 %**, y sale de la estructura de costos, no de los datos. El monto multiplica a toda la cuenta, así que cambia cuánto ganamos o perdemos pero no mueve el umbral. Esto es importante por dos motivos. Primero, fija un criterio de decisión estable y defendible probablemente sin necesidad de tunear nada. Segundo, condiciona qué pedirle al modelo: lo que necesitamos no es una clasificación, sino una probabilidad de fraude bien calibrada alrededor de 0,20, porque ahí es donde se toma la decisión. Esta es la hipótesis de trabajo que recorre todo el informe: si las probabilidades representan bien el riesgo real y los costos son los enunciados, aprobar por debajo de 0,20 es óptimo, y el resto del proyecto consiste en conseguir el modelo que ordena y calibra mejor el riesgo sin romper ese criterio.

2. Análisis y transformaciones del dataset

El dataset trae las features anonimizadas, de la **a** a la **p** (sin la **i**), más **fecha**, **monto**, **score** y el target **fraude**. Sin semántica de las columnas, el tipo de cada una (binaria, categórica, discreta, continua) se deduce de su contenido. Todo el análisis se hace sobre el conjunto de desarrollo (dev) y deja afuera la última semana, reservada como test, para no sesgar las decisiones que vienen después.

2.1. Calidad de datos, faltantes y leakage

Los faltantes no son ruido, traen información. La columna **o** falta en el 72 % de las filas, y cuando falta la tasa de fraude es del 2 %, la mitad de la base, cuando está presente sube a cerca del 13 %, más del doble. La ausencia de **o** es **informativa**, así que la decisión natural es tratarla como un nivel más y no imputarla. Otras columnas comparten patrón de ausencia (**b** y **c** faltan juntas en el 10,5 % de los casos, **d** con **m**, **f** con **l**), lo que sugiere un mismo origen de proceso, de ahí salen indicadores de faltante por bloque.

k merece un párrafo propio porque su unicidad podría esconder problemas: tiene un valor distinto por fila, repartido de forma casi uniforme en $[0, 1]$. Eso obliga a chequear duplicados con y sin

k, y no aparecen filas repetidas en ninguno de los dos casos. **k** no correlaciona con el target y queda marcada como sospechosa de ser ruido o identificador (por sus valores no parecería un identificador), a confirmar en modelado.

El **score** es la feature más delicada por el lado del leakage. Tiene una relación fuerte con el fraude, sobre todo en valores altos, y podría parecer el score de un sistema de prevención previo. Si fuera así, hay que confirmar que está disponible antes de decidir si se aprueba la operación: si se calculara después, usarlo sería leakage. Como no podemos garantizarlo desde los datos y se desconoce su origen, el proyecto entero se divide en dos escenarios, con **score** y sin él, para medir cuánto del desempeño depende de esa feature.

2.2. Distribuciones, outliers y la transformación que cada modelo necesita

monto, **c**, **e** y **f** están muy sesgadas a la derecha. En escala logarítmica **monto** se vuelve casi simétrico (Figura 1), y **e** muestra una masa grande en cero más un cuerpo continuo, lo que se parece a una mezcla de un cero y un valor. **f** llega a valores negativos, así que la transformación que conserva el signo y aplica $\log 1p$ sobre el valor absoluto es la adecuada.

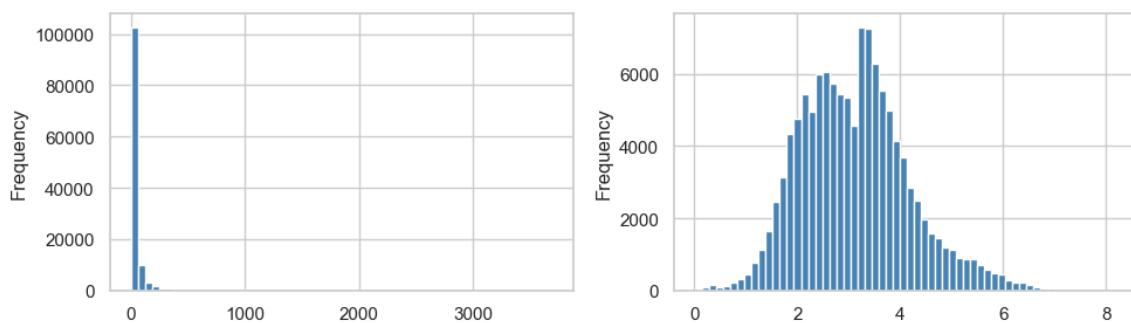


Figura 1: Distribución de **monto** en el conjunto de desarrollo. A la izquierda se muestra la escala original, dominada por una cola larga de montos altos; a la derecha, la transformación $\log(1 + \text{monto})$, que permite ver con mayor claridad el cuerpo de la distribución.

La regla del IQR marca como extremos alrededor del 11 % de los valores en **c** y **f**, con colas larguísimas (**c** supera los 13 millones, **f** llega a 145 mil). Con miles de casos marcados y no unos pocos puntos sueltos, llamarlos errores y eliminarlos sería apresurado: son parte de la distribución. Por eso no se recorta ni se elimina nada. El caso de **monto** es además económico: el grupo de montos altos tiene una tasa de fraude de 8,9 % contra 4,8 % en el resto, así que esas operaciones concentran riesgo y peso a la vez.

De acá sale una decisión de feature engineering que separa los tipos de modelo. La regresión logística recibe logs, escalado, indicadores de faltante, el indicador de **e**=0 y las variables derivadas de **fecha**. Los modelos de árbol reciben las variables numéricas sin transformaciones de escala. XGBoost, LightGBM y CatBoost utilizan su soporte nativo para faltantes y categóricas. Random Forest usa imputación por mediana, indicadores de ausencia, one-hot para las categóricas de baja cardinalidad y target encoding para **j**.

2.3. La categórica de alta cardinalidad

j tiene 7.898 niveles y muchos aparecen una sola vez. Un one-hot completo no es práctico, así que el encoding se elige por modelo. La logística agrupa los niveles infrecuentes (los de menos de 20 apariciones) en una sola columna. Los boosting manejan la categórica de forma nativa. Random Forest, que necesita una matriz densa, recibe **j** por target encoding con *cross-fitting* ($cv=5$), que evita que cada fila vea su propia etiqueta. En todos los casos los niveles se aprenden *solo en el*

train de cada fold y los no vistos en validación van a una categoría de descarte, porque mirar la validación para definir el encoding sería leakage. También se probó target encoding en la logística, pero quedó descartado: ordena algo mejor (PR-AUC) pero deja menos ganancia al umbral de 0,20 (Tabla 6). El análisis temporal muestra que entre el 11 % y el 13 % de las categorías de *j* en validación son nuevas, pero representan apenas el 2 % o 3 % de las transacciones: la mayoría del volumen cae en categorías conocidas.

2.4. Análisis temporal

La composición de las operaciones cambia a lo largo de los 38 días de desarrollo (dev). El **score** medio se mantiene cerca de 51 las primeras semanas y baja a 45 desde la quinta, la mediana de **monto** también baja (Véase Apéndice Figura 7). La tasa de fraude se mueve día a día sin un patrón claro en una ventana tan corta. Esto tiene una consecuencia directa: validar mezclando fechas al azar filtraría futuro en el train. La validación tiene que respetar el tiempo, entrenando con el pasado y midiendo sobre el futuro.

3. Esquema de validación

La validación se diseña antes que el modelo. El conjunto de test es la última semana completa (15 al 21 de abril, cerca del 19 % de los datos) y queda bloqueado hasta la evaluación final. Sobre el desarrollo (dev) se arman tres folds temporales con *train* acumulado: el inicio del train queda fijo y cada fold valida la semana siguiente (Tabla 1). El esquema, sus máscaras y los chequeos de integridad viven en código y fallan si algún fold se superpone, invierte el orden temporal o toca el test.

Cuadro 1: Folds temporales de validación (train acumulado). El test queda fuera. Para una interpretación más visual, véase también Apéndice figura 8

Fold	Train	Validación	Prevalencia val.
1	08/03 – 25/03	25/03 – 01/04	6,0 %
2	08/03 – 01/04	01/04 – 08/04	5,6 %
3	08/03 – 08/04	08/04 – 15/04	5,2 %

Son tres folds y no más porque con 38 días la alternativa fuerza trains iniciales demasiado cortos o validaciones de menos de una semana, que introducen sesgo de día de semana, es una decisión de diseño. El esquema completo, con el test bloqueado al final, está en la Figura 8 del apéndice. El train es en principio acumulado y no deslizante porque descartar historia con tan pocos datos penalizaría a los folds tempranos, la ventana deslizante se reserva como prueba de robustez para los finalistas, no como esquema base. La prevalencia y la dificultad cambian entre folds, así que la comparación nunca se hace sobre el mejor fold sino sobre la ganancia acumulada y su estabilidad.

3.1. Cómo se agregan las métricas entre folds

Las métricas se calculan por separado en cada fold para respetar la estructura temporal. Cada validación representa una semana distinta y puede tener diferente cantidad de operaciones, prevalencia de fraude y distribución de features. La ganancia se suma entre folds porque es una magnitud económica aditiva. Las métricas predictivas como ROC-AUC, PR-AUC y Brier no responden de la misma manera cuando cambia la prevalencia de fraude. En estos datos, los tres folds tienen prevalencias de 6,0 %, 5,6 % y 5,2 %.

ROC-AUC. La curva ROC utiliza la tasa de verdaderos positivos y la tasa de falsos positivos. El TPR mide qué proporción de los fraudes fue detectada. El FPR mide qué proporción de las operaciones legítimas fue rechazada incorrectamente.

En ambos casos, el denominador pertenece a una sola clase. El TPR se calcula únicamente sobre los fraudes y el FPR únicamente sobre las operaciones legítimas. Por eso, si cambia solamente la proporción entre ambas clases, pero el modelo mantiene el mismo comportamiento dentro de cada una, estas tasas no cambian. Por este motivo, los ROC-AUC de los distintos folds se promedian directamente.

Esto no significa que ROC-AUC sea estable en el tiempo, puede cambiar si cambia la distribución de scores dentro de los fraudes o dentro de las legítimas. El promedio solo resume el ordenamiento observado en las distintas semanas

En un problema desbalanceado, ROC-AUC resulta optimista porque el FPR se calcula sobre una gran cantidad de operaciones legítimas y evalúa todo el rango de umbrales, incluso regiones no relevantes. Aun así, se reporta como métrica auxiliar no solo para ver esta diferencia con PR-AUC sino también porque resume la capacidad global de discriminación del modelo y facilita la comparación de su ordenamiento entre folds. No se utiliza de forma aislada.

PR-AUC. La curva Precision-Recall utiliza recall, que es equivalente al TPR, y precision. A diferencia del FPR, el denominador de precision mezcla las dos clases. Incluye tanto los fraudes correctamente detectados como las operaciones legítimas rechazadas por error.

Si se duplica la cantidad de operaciones legítimas y el modelo mantiene la misma tasa de falsos positivos, también se duplica aproximadamente la cantidad de falsos positivos. El FPR permanece igual porque su numerador y su denominador aumentan en la misma proporción. En cambio, precision disminuye porque aparecen más falsos positivos junto a la misma cantidad de fraudes detectados.

Por eso PR-AUC depende directamente de la prevalencia. En un ranking aleatorio, la precision esperada es aproximadamente igual a la proporción de fraudes del conjunto:

$$\text{PR-AUC}_{\text{base}} \approx \pi. \quad (3)$$

Esto implica que un PR-AUC de 0,30 no representa la misma mejora si la prevalencia es 2% que si es 15%. El valor puede promediarse matemáticamente, pero el promedio crudo mezclaría folds con diferentes puntos de partida.

Para comparar el desempeño relativo contra la referencia de cada semana uso:

$$\text{PR-AUC}_{\text{norm}} = \frac{\text{PR-AUC} - \pi}{1 - \pi}. \quad (4)$$

La resta elimina la referencia propia del fold y la división por $1 - \pi$ expresa cuánto del recorrido entre esa referencia y el resultado perfecto fue cubierto por el modelo. Un valor de 0 representa un ranking aleatorio para la prevalencia del fold, 1 representa un ranking perfecto y un valor negativo indica un resultado peor que la referencia.

Brier Score. Brier mide el error cuadrático medio de las probabilidades predichas:

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2, \quad (5)$$

donde p_i es la probabilidad de fraude estimada para la operación i , y_i vale 1 si la operación es fraudulenta y 0 si es legítima, y N es la cantidad de operaciones evaluadas.

Un Brier menor indica que las probabilidades predichas están más cerca de los resultados observados. Una predicción incorrecta y muy segura recibe una penalización mayor. Por ejemplo, asignar una probabilidad de fraude de 0,90 a una operación legítima produce un error de $(0,90 - 0)^2 = 0,81$, mientras que asignarle 0,10 produce $(0,10 - 0)^2 = 0,01$.

Para interpretar Brier hace falta una referencia. Considero un predictor base que no intenta distinguir las operaciones y asigna a todas la prevalencia de fraude del fold:

$$p_i = \pi. \quad (6)$$

Una operación es fraude con probabilidad π . En ese caso, su error cuadrático es $(\pi - 1)^2 = (1 - \pi)^2$. Una operación es legítima con probabilidad $1 - \pi$ y su error es $(\pi - 0)^2 = \pi^2$. El error esperado del predictor base es entonces:

$$\begin{aligned} \text{Brier}_{\text{base}} &= \pi(1 - \pi)^2 + (1 - \pi)\pi^2 \\ &= \pi(1 - \pi). \end{aligned} \quad (7)$$

Esta referencia cambia con la prevalencia. Cuando el fraude es poco frecuente, un predictor que asigna probabilidades bajas a todas las operaciones puede obtener un Brier aparentemente pequeño sin discriminar entre fraudes y operaciones legítimas.

Por eso, comparar directamente el Brier de folds con distintas prevalencias puede resultar engañoso. Para medir la mejora respecto del predictor base uso Brier Skill Score:

$$\text{BSS} = 1 - \frac{\text{Brier}}{\pi(1 - \pi)}. \quad (8)$$

Un BSS de 0 significa que el modelo obtiene el mismo Brier que asignar la prevalencia del fold a todas las operaciones. Un BSS de 1 corresponde a probabilidades perfectas y un valor negativo indica que el modelo es peor que el predictor base.

Resumen de la agregación. La ganancia se suma entre folds porque representa valor económico acumulado. ROC-AUC se promedia directamente porque su referencia de 0,5 no cambia con la prevalencia. PR-AUC y Brier también se reportan en sus valores por semana. Sin embargo, para resumirlos entre folds se promedian PR-AUC normalizado y Brier Skill Score.

Estas métricas son auxiliares. ROC-AUC y PR-AUC describen el ordenamiento del riesgo, mientras que Brier evalúa la calidad de las probabilidades. La selección principal del modelo continúa basada en la ganancia obtenida con el umbral económico de 0,20.

4. Modelos utilizados

4.1. Baseline: políticas y regresión logística

Antes de cualquier ensamble conviene saber contra qué se compara. Aprobar todo deja una ganancia de referencia que el modelo debe superar, rechazar todo da cero. Una tercera referencia usa el `score` como regla directa, eligiendo en cada train el corte que habría dejado más ganancia y aplicándolo a la validación siguiente. Esa regla supera a aprobar todo, pero el corte óptimo se mueve de fold a fold (98, 95, 93), lo que anticipa que una regla fija sobre el `score` podría degradarse con el tiempo.

La regresión logística es la primera referencia entrenada, elegida por interpretable. Con el pre-procesamiento justificado en el EDA, supera a las políticas simples (Tabla 2). El salto respecto de aprobar todo es grande, mientras que la regla de **score** aporta poco. La versión sin **score** sigue siendo competitiva en dev, por lo que se mantienen los dos escenarios.

Cuadro 2: Baseline sobre las tres validaciones temporales de desarrollo. La ganancia está en las unidades de la función objetivo y la mejora es relativa a aprobar todo.

Estrategia	Ganancia	Mejora	Fraudes rechazados
Aprobar todo	317.028	ref.	0 %
Regla de score	346.605	+9,3 %	19,6 %
Logística sin score	413.696	+30,5 %	45,0 %
Logística con score	420.516	+32,6 %	48,0 %

La logística también permitió cerrar tres decisiones de feature engineering. **k** recibe un coeficiente pequeño y no mejora la ganancia. El target encoding de **j** mejora algunas métricas de probabilidad, pero deja menos ganancia al umbral de 0,20. El rebalanceo desplaza la escala **y**, aun después de recalibrar, queda por debajo de la referencia. Por eso se conservan el one-hot agrupado y el modelo sin rebalanceo.

4.2. Modelos de árbol y búsqueda de hiperparámetros

La fase de boosting compara Random Forest, XGBoost, LightGBM y CatBoost, manteniendo los dos escenarios y la ganancia como objetivo. Para evitar evaluar sobre los mismos períodos usados para seleccionar hiperparámetros, aplico una **validación temporal anidada**. Para evaluar en el fold 2, Optuna selecciona la configuración usando el fold 1 como validación interna. Para evaluar en el fold 3, utiliza los folds 1 y 2. Una vez elegida la configuración, el modelo se reentrena con todo el historial anterior al fold externo y se evalúa sobre ese período, que no participó de la búsqueda.

Optuna optimiza directamente la ganancia (no el AUC), con un sampler TPE sembrado y un *pruner* por mediana, sobre los espacios habituales de cada familia (profundidad, learning rate, regularización, submuestreo). El TPE concentra los trials en las regiones que vienen rindiendo mejor, más eficiente que una búsqueda aleatoria con el mismo presupuesto, y el pruner corta temprano los trials que quedan por debajo de la mediana de los anteriores, para no gastar cómputo en candidatos que ya pintan peores. Los estudios se guardan en SQLite y se reanudan sin repetir trials, lo que hace la corrida reproducible y barata de retomar. Optimizar sobre la métrica de negocio es deliberado: como la decisión final es económica, preferimos el modelo que más gana al umbral fijo antes que el de mejor AUC.

Sobre los dos folds externos, los tres boosting con **score** quedan muy cerca entre sí y por encima de la logística (Tabla 3). Cada ajuste externo se repite con tres semillas (42, 43 y 44) para separar la señal del ruido de inicialización: las cifras que se reportan son el promedio, y la variación entre semillas puede medir cuándo una diferencia es real o es ruido. LightGBM logra la mayor ganancia, CatBoost el mejor ordenamiento y calibración, XGBoost queda en el medio. Random Forest no muestra una ventaja propia y se decide descartarlo. Sin **score** el orden cambia y CatBoost pasa a ser el mejor (Tabla 4). Como solo hay dos folds externos y las diferencias son chicas, los tres boosting con **score** más CatBoost sin **score** pasan a una validación más exigente en vez de cerrar la elección acá.

Cuadro 3: Boosting sobre los folds externos 2 y 3, escenario con **score**. Ganancia sumada, ordenamiento y calibración en sus versiones normalizadas.

Modelo	Ganancia	ROC-AUC	PR-AUC norm.	Brier Skill
LightGBM	307.576	0,878	0,436	0,260
CatBoost	305.633	0,889	0,458	0,278
XGBoost	305.046	0,888	0,452	0,270
Random Forest	302.934	0,881	0,431	0,258
Logística (ref.)	301.891	0,842	0,341	0,193

Cuadro 4: Boosting sobre los folds externos 2 y 3, escenario sin **score**. Mismo formato que la Tabla 3. Acá CatBoost gana en todas las columnas.

Modelo	Ganancia	ROC-AUC	PR-AUC norm.	Brier Skill
CatBoost	297.380	0,854	0,377	0,216
Random Forest	295.723	0,837	0,342	0,181
LightGBM	295.498	0,842	0,353	0,196
XGBoost	290.983	0,848	0,360	0,202
Logística (ref.)	296.766	0,841	0,303	0,165

4.3. Desbalance, reponderación y calibración

El proyecto no usa SMOTE ni pesos de clase, y la decisión está respaldada por experimentos. Hay miles de fraudes en cada train, suficientes para separar, y lo que necesitamos son probabilidades con interpretación económica alrededor de 0,20, que es lo que el rebalanceo rompe. La pregunta se cierra con un test parejo sobre los boosting: reponderar con **scale_pos_weight** y después recalibrar para devolver la escala. El modelo reponderado pierde ganancia al umbral fijo por el corrimiento de escala. Al recalibrar, la ganancia se recupera en gran parte, pero el ordenamiento no mejora: el PR-AUC normalizado de CatBoost baja de 0,457 a 0,438 y el de XGBoost de 0,452 a 0,215. La reponderación no mejora la referencia y se descarta.

La calibración como chequeo en sí (sin reponderado) da el resultado inverso: las probabilidades crudas de los boosting ya tienen escala razonable. Aplicar un calibrador sigmoide mueve la ganancia menos de 1.500 y el Brier menos de 0,0003, con signo mezclado. Los modelos quedan en un Brier de 0,037 a 0,042 y con Brier Skill Score positivo en todos los casos. Casi no cambian al calibrar. Se sigue con las probabilidades originales.

5. Robustez y selección del modelo final

A partir de esta etapa, la selección principal se concentra en los cuatro modelos con **score**: regresión logística, XGBoost, LightGBM y CatBoost. CatBoost y la regresión logística sin **score** se mantienen como alternativas separadas por si esa feature no estuviera disponible, pero no compiten directamente con modelos que utilizan más información. Con diferencias pequeñas entre los candidatos principales, comparo cuánto depende cada uno del historial usado para entrenar: train acumulado contra una ventana móvil de 17 días, manteniendo las mismas semanas de validación. Se eligen 17 días porque coincide con la historia disponible en el primer train y permite comparar ventanas de tamaño similar entre folds.

Cuadro 5: Ganancia con **score** al pasar de train acumulado a ventana de 17 días (folds 2 y 3, promedio de tres semillas). Entre paréntesis, el cambio.

Modelo	Acumulado	Ventana	Cambio
CatBoost	306.470	305.675	−795
XGBoost	305.046	305.895	+849
LightGBM	307.576	301.286	−6,290
Logística	301.891	299.716	−2,174

El resultado (Tabla 5) ordena la decisión. CatBoost casi no cambia entre acumulado y ventana, XGBoost mejora un poco, y **LightGBM pierde unos 6.300**. Su ventaja resulta sensible al historial disponible, por lo que queda fuera pese a haber liderado en ganancia con el train acumulado. Cuando dos opciones empatan dentro del ruido, CatBoost mantiene un rendimiento comparable usando datos más recientes. Por eso se elige la ventana ante los cambios de composición observados. El finalista es **CatBoost con score sobre ventana de 17 días**, con XGBoost como respaldo y CatBoost sin **score** como alternativa si esa feature no estuviera disponible.

Antes de cerrar se probaron varias ideas de mejora, y ninguna se sostuvo:

- **Features de frecuencia de j** (frecuencia, rareza, conteo de faltantes): no mejoran la ganancia ni el PR-AUC normalizado.
- **Mezcla de CatBoost y XGBoost**: queda dentro de la variación entre semillas y no justifica mayor complejidad.
- **Segmentación**: los modelos separados por monto alto o $o=N$ dejan menos ganancia. El híbrido por familia no se entrena porque la señal por segmento no alcanza

Las categorías nuevas de j representan entre 2% y 3% de las validaciones. En esta muestra tienen 3,7% de fraude frente a 5,5% en las conocidas y dejan ganancia positiva en todos los modelos. El resultado es descriptivo por el tamaño del grupo.

La Tabla 6 resume los cambios evaluados. Ninguno mejora la ganancia de forma estable al umbral de 0,20.

Cuadro 6: Experimentos evaluados y descartados, con el motivo en cada caso.

Experimento		Qué se probó	Por qué se descartó
Feature k (logística y árboles)		Incluirla por si aporta sola o en interacciones	No muestra una mejora estable en las familias evaluadas
Target encoding de j (logística)		Reemplazar el one-hot agrupado	Mejora AUC y calibración pero deja menos ganancia al umbral de 0,20
<code>class_weight=balanced</code> (logística)		Rebalanceo de clases	Hunde la ganancia, y recalibrar repara la escala pero no el orden
<code>scale_pos_weight</code> (boosting)		Reponderar la clase de fraude	Igual que el rebalanceo: empeora el ordenamiento (PR-AUC y ROC-AUC)
Calibración (boosting)	sigmoide	Recalibrar las probabilidades	No mejora Brier ni ganancia de forma estable
Features de frecuencia de j		<code>j_frecuencia</code> , rareza y conteo de faltantes	No mejoran la ganancia ni el PR-AUC normalizado
Mezcla Boost + XGBoost	Cat-	Promedio ponderado de ambos	La mejora cae dentro del ruido entre semillas, no paga la complejidad
Segmentación por monto alto		Un modelo aparte para el 10 % de montos más altos	Menor ganancia, PR-AUC normalizado y Brier Skill Score que el modelo único.
Segmentación por $o=N$		Un modelo aparte para el segmento más riesgoso	Menos ganancia que el modelo único entrenado con todo
Híbrido por segmento		Usar CatBoost en unos segmentos y XGBoost en otros	La señal observada en $o=N$ no alcanza en principio para justificar otra arquitectura

6. Interpretabilidad

CatBoost combina árboles, relaciones no lineales e interacciones, para entender cómo construye sus predicciones uso SHAP. Para cada operación, SHAP asigna un valor a cada feature que representa cuánto mueve la salida del modelo respecto de una predicción base. Ese aporte se calcula comparando la predicción con y sin la feature disponible y promediando su contribución sobre las distintas combinaciones posibles de las demás features.

La Figura 2 global muestra el promedio del valor absoluto de SHAP para cada feature. Se utiliza el valor absoluto porque una misma feature puede empujar algunas operaciones hacia fraude y otras hacia legítima. Si se promediaran directamente los valores positivos y negativos, ambos efectos podrían cancelarse. Por lo tanto, una barra más alta indica que la variable suele producir cambios de mayor magnitud en la salida del modelo, pero no muestra la dirección.

Los valores SHAP se calculan sobre una muestra de 3.000 operaciones de la ventana de entrenamiento de 17 días.

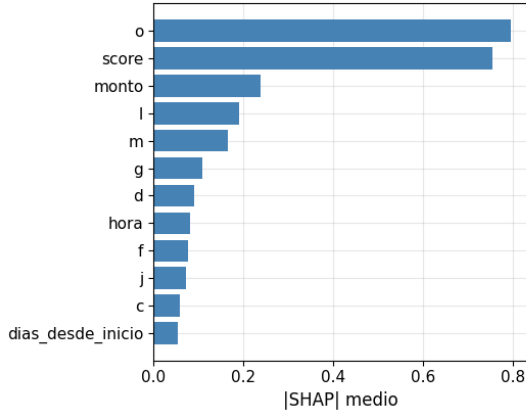


Figura 2: Importancia global del CatBoost final, medida como el valor SHAP absoluto medio sobre una muestra de la ventana de entrenamiento. Valores más altos indican mayor impacto promedio en la predicción; `o` y `score` concentran la mayor parte de la señal.

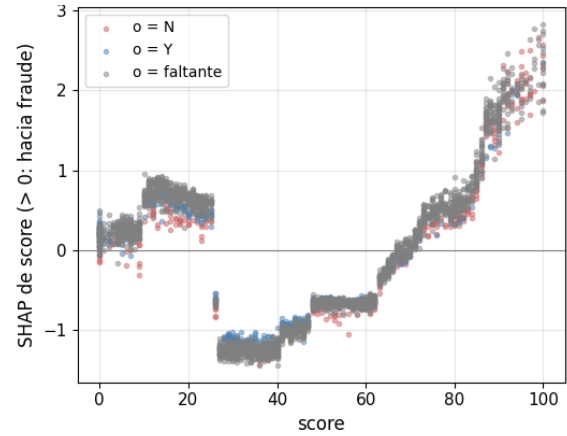


Figura 3: Contribución SHAP de `score` según su valor y el nivel de `o`. Cada punto representa una operación; contribuciones positivas empujan la predicción hacia fraude. El efecto aumenta en scores altos y es más marcado cuando `o=N`.

En la lectura, `o` y `score` concentran los principales aportes, seguidos por `monto`, en línea con los patrones observados en el EDA. `j`, que en la regresión logística aparecía distribuida entre coeficientes one-hot (ver Notebook 03), tiene menor importancia global en CatBoost. A diferencia de la logística, CatBoost procesa las features categóricas de forma nativa mediante estadísticas basadas en target y frecuencia, sin crear una columna por cada categoría. Por eso, la importancia de `j` no es directamente comparable entre ambos modelos, ya que cada uno representa y utiliza esta feature de una manera diferente.

En la Figura 3, cada punto representa una operación. En el eje horizontal aparece el valor de `score` y en el vertical su contribución SHAP. Se puede observar que `score` aporta poco en valores bajos y empuja con fuerza hacia fraude en valores altos. Para un mismo score, las operaciones con `o=N` reciben una contribución mayor que las de `o=Y` o con `o` faltante. Esto muestra que el modelo utiliza ambas features de forma combinada, en línea con la interacción observada durante el desarrollo.

7. Evaluación final

La medición definitiva se hace una sola vez sobre la semana de test bloqueada, con el modelo congelado y promediando tres semillas. CatBoost supera a la logística y a aprobar todo (Tabla 7 y Figura 4).

Cuadro 7: Evaluación sobre el test bloqueado (promedio de tres semillas).

Modelo	Ganancia	ROC-AUC	PR-AUC	Brier
CatBoost (acumulado)	227.914	0,887	0,434	0,031
CatBoost (ventana)	223.882	0,883	0,426	0,031
Logística (ventana)	218.835	0,838	0,327	0,034
Aprobar todo	191.247	n/d	n/d	n/d

CatBoost queda por encima de aprobar todo y de la regresión logística, aunque todavía existe una diferencia importante respecto del máximo teórico (284.827). Este máximo funciona como refe-

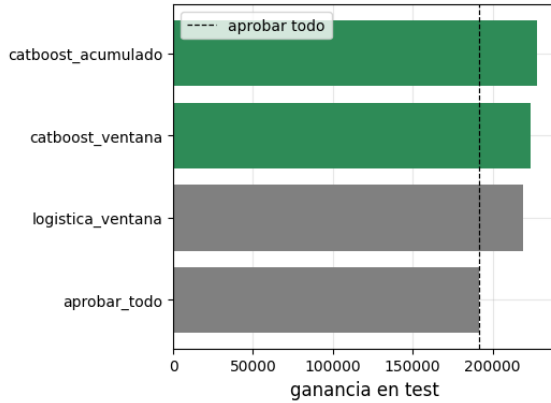


Figura 4: Ganancia obtenida por cada estrategia sobre la semana de test bloqueada. Se comparan CatBoost con ventana de 17 días y train acumulado, la regresión logística con ventana y la política de aprobar todo. Para CatBoost se reporta el promedio de tres semillas.

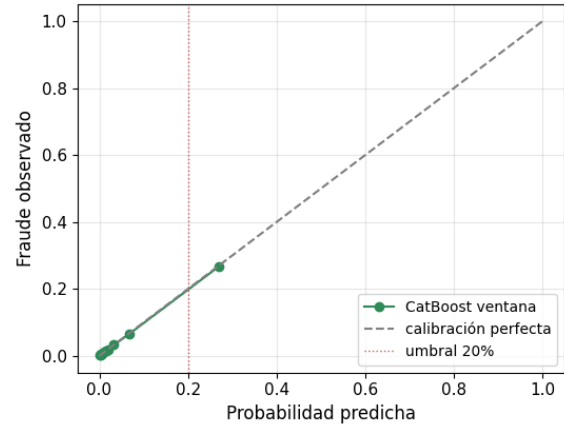


Figura 5: Calibración del CatBoost final sobre el test bloqueado. Las operaciones se agrupan en diez grupos de igual tamaño y se compara la probabilidad media predicha con la tasa de fraude observada. La diagonal representa calibración perfecta y la línea vertical marca el umbral económico de 0,20.

rencia y no como un resultado alcanzable, ya que supone conocer perfectamente qué operaciones son fraudulentas.

Para evaluar la calibración, ordeno las operaciones por la probabilidad predicha y las divido en diez grupos de igual tamaño. En cada decil comparo la probabilidad media estimada con la tasa de fraude observada. La diagonal representa calibración perfecta: los puntos por encima indican que el modelo subestima el riesgo y los puntos por debajo que lo sobreestima. La curva se mantiene cerca de la diagonal y no muestra un corrimiento general de las probabilidades en test (Figura 5). El agrupamiento en deciles es un diagnóstico general y no una medición precisa exactamente sobre 0,20. El umbral se mantiene como la regla económica derivada de los costos.

CatBoost obtiene mayor ganancia que la logística, mientras que la logística es más simple, rápida y fácil de interpretar. Si se prioriza la ganancia, elegiría CatBoost. Si la simplicidad tiene mayor peso, la regresión logística sigue siendo una alternativa razonable.

8. Llevar el modelo a producción - Preguntas 3, 4 y 5

8.1. ¿Qué pasos puedo seguir para intentar asegurar que la performance del modelo en laboratorio será similar a la de producción?

Lo primero es tratar de que la evaluación de laboratorio se parezca lo más posible al uso real. Por eso hice la división por fecha: aunque no conocemos la dinámica real de producción o la ventana de evaluación, el modelo entrena con operaciones pasadas y se evalúa con operaciones posteriores. También dejé una semana de test sin mirar hasta el final y ajusté las transformaciones solamente con el train de cada fold. Esto reduce el riesgo de obtener un buen resultado por leakage o haber mezclado pasado y futuro.

Antes de llevarlo a producción confirmaría que todas las features se puedan calcular de la misma forma y en el mismo momento. Esto es importante por ejemplo para `score`, porque es una de las features que más usa el modelo y metodológicamente no confirmamos si está disponible. Si no lo estuviera, validaría de forma final la alternativa sin `score` evaluada durante el trabajo.

También tendría en cuenta que la etiqueta de fraude puede tardar en confirmarse en producción.

No evaluaría ni reentrenaría con períodos muy recientes si sus etiquetas todavía pueden cambiar. La métrica principal debería seguir siendo la misma ganancia económica usada en el challenge, no solamente el AUC.

8.2. Suponiendo que la performance predictiva en producción es muy diferente a la esperada, ¿cuáles cree que son las causas más probables?

Una causa probable es el *data drift*: que cambie el tipo de operaciones que recibe el modelo. En el EDA ya se observa que `score` y `monto` se mueven durante el período disponible. Si ese cambio continúa, los datos de producción pueden ser distintos a los usados para entrenar. También puede aparecer *concept drift*, es decir, que cambie la relación entre las features y el fraude. Los atacantes se adaptan a las reglas del sistema, por lo que una señal que hoy funciona puede perder fuerza.

Otra posibilidad es una diferencia entre el pipeline de entrenamiento y el de producción. Por ejemplo, que una feature se calcule en otro momento, que los faltantes se representen de otra manera o que una categoría nueva de `j` no reciba el mismo tratamiento. Un cambio de escala, definición o disponibilidad de `score` tendría un impacto grande.

Por último, la diferencia puede estar en la medición y no necesariamente en las predicciones. Las labels de producción podrían estar incompletas, tener una definición distinta o depender de qué operaciones terminan observándose bajo la política existente. Antes de atribuir la caída al modelo, revisaría la cobertura de las labels y la calidad de los datos utilizados para calcular las métricas.

8.3. ¿Qué pasos debería seguir para poner el nuevo modelo en producción?

Primero integraría la solución con la infraestructura de ML existente en la empresa. Versionaría el modelo, la construcción de features, los niveles categóricos, las dependencias y el umbral de 0,20. Si existe un feature store, reutilizaría las mismas definiciones para entrenamiento e inferencia. Si no, mantendría una implementación compartida y pruebas de consistencia entre ambos entornos.

El mecanismo de serving dependería de los estándares y requisitos de latencia. Podría utilizar un contenedor de Docker a través de una API o la plataforma interna disponible. Después de las pruebas de integración, comenzaría con una parte chica del tráfico y aumentaría esa proporción gradualmente, manteniendo una versión anterior por seguridad.

En producción monitorearía tanto features técnicas como de negocio: latencia, errores, disponibilidad de features, drift en sus distribuciones, categorías nuevas, tasa de aprobación, fraude aprobado y ganancia. Usaría una herramienta como MLflow para registrar los datos, parámetros, métricas y artefactos de cada versión. Finalmente definiría una frecuencia de reentrenamiento con datos recientes y criterios para decidir cuándo alcanza con recalibrar y cuándo hace falta entrenar una nueva versión.

9. Conclusión

El modelo final es un CatBoost con `score`, entrenado sobre una ventana móvil de 17 días, con las features base, sin reponderación, recalibración ni segmentación, y decidiendo al umbral económico de 0,20. En el test bloqueado obtiene una ganancia de 223.882, frente a 218.835 de la regresión logística y 191.247 de aprobar todo.

La ganancia fue la métrica principal, incluso durante la búsqueda de hiperparámetros. La validación respetó el orden temporal y el test no participó de la selección. El ordenamiento y la calidad de las probabilidades se analizaron por separado. La ventana se eligió en desarrollo por

ofrecer un rendimiento comparable con datos más recientes, aunque en el test el entrenamiento acumulado obtuvo una ganancia mayor.

Antes de producción deben validarse la disponibilidad de **score**, la madurez y definición de las etiquetas y posibles sesgos de selección. Si **score** no estuviera disponible, CatBoost sin score y la regresión logística sin score, ambos evaluados en desarrollo, requerirían una validación final antes del despliegue.

A. Material complementario

Las figuras complementan el análisis univariado, temporal y de validación.

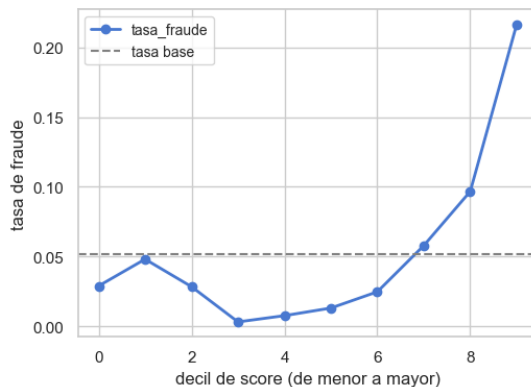


Figura 6: Tasa de fraude por decil de **score** en desarrollo. La línea horizontal indica la prevalencia global; aunque la relación no es perfectamente monótona, el último decil presenta mayor riesgo.

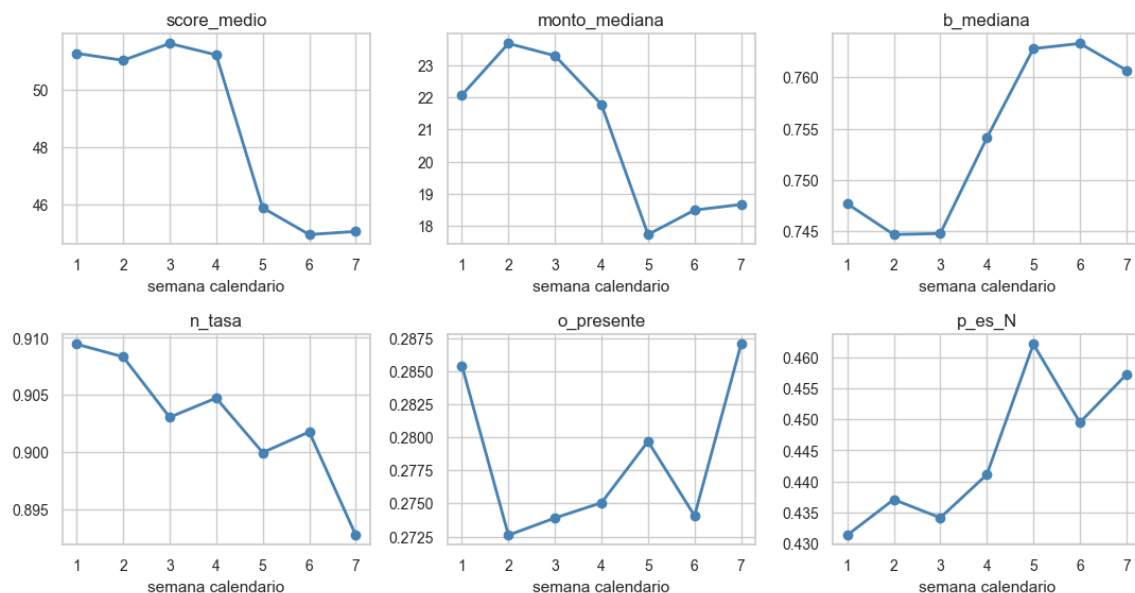


Figura 7: Evolución semanal de seis features durante el período de desarrollo. Se muestran la media de **score**, las medianas de **monto** y **b**, y las proporciones de **n=1**, **o** presente y **p=N**. Los cambios en **score** y **monto** respaldan la validación temporal.

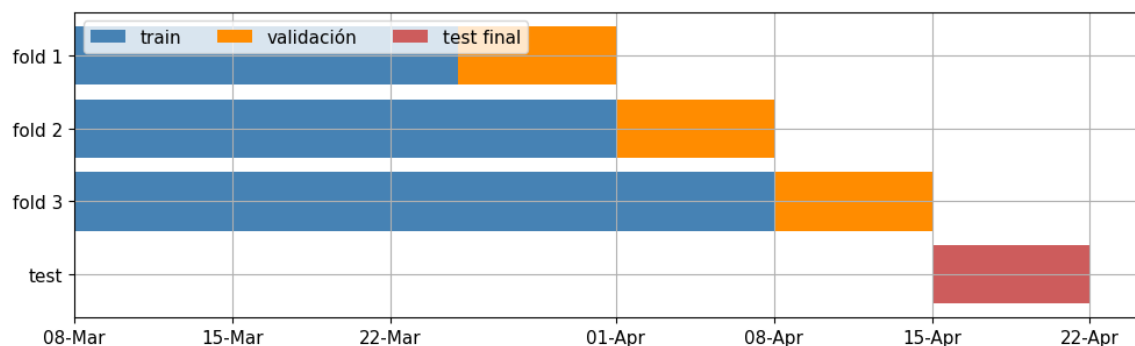


Figura 8: Esquema temporal de evaluación. Cada fold entrena con el historial disponible y valida sobre los siete días siguientes. La última semana queda separada como test final y no participa del ajuste ni de la selección.